



ESTUDIO
SHONOS

SISTEMA DE GESTION DE PROCESOS





CONTENIDOS

- 01. Introducción
- 02. Puntos clave
- 03. Tecnologías
- 04. Beneficios
- 06. Comparativas
- 07. Alternativas
- 08. Visión global
- 10. Conclusiones

CAPITULO 1



1.1 DESCRIPCIÓN DEL PROBLEMA

Los sistemas operativos modernos necesitan administrar múltiples procesos simultáneamente para funcionar con rapidez y estabilidad. Una gestión ineficiente de estos elementos provoca graves problemas en el entorno computacional, tales como tiempos de espera elevados, consumo innecesario de recursos y una desorganización crítica en el uso del CPU.

Para solucionar esto, se desarrolló un Sistema de Gestión de Procesos (SGP) en C++ utilizando estructuras de datos dinámicas (nodos y punteros) en lugar de arreglos fijos. Este enfoque resuelve directamente los problemas de la ausencia de un registro flexible, la planificación ineficiente del procesador y el desorden al asignar o liberar bloques de memoria en tiempo de ejecución.

Para solucionar esto, se desarrolló un Sistema de Gestión de Procesos (SGP) en C++ utilizando estructuras de datos dinámicas (nodos y punteros) en lugar de arreglos fijos. Este enfoque resuelve directamente los problemas de la ausencia de un registro flexible, la planificación ineficiente del procesador y el desorden al asignar o liberar bloques de memoria en tiempo de ejecución.

1.2 REQUERIMIENTOS DEL SISTEMA

Los requerimientos del sistema se dividen en funcionales y no funcionales, los funcionales son todas las acciones y operaciones que realizará el sistema durante su ejecución, mientras que los no funcionales son las condiciones y características técnicas que el programa deberá cumplir para funcionar de manera correcta, ordenada y estable durante su uso.

REQUERIMIENTOS FUNCIONALES

1. MÓDULO GESTOR DE PROCESOS

RF-01 El sistema permitirá registrar nuevos procesos almacenando datos como el ID, nombre, prioridad y estado de cada proceso.

RF-02 También permitirá eliminar procesos cuando ya no sean necesarios dentro del sistema.

RF-03 El usuario podrá buscar procesos mediante su ID o nombre para visualizar toda su información registrada.

3. Módulo Gestor de Memoria

RF-09 El sistema permitirá asignar bloques de memoria utilizando operaciones push sobre una pila.

2. Módulo Planificador de CPU

RF-06 El sistema organizará los procesos dentro de una cola según su nivel de prioridad para mantener un mejor orden de ejecución.

RF-07 También ejecutará procesos mediante operaciones de descolamiento respetando el orden establecido dentro de la cola.

4. Interfaz y Persistencia

RF-12 El sistema contará con un menú interactivo por consola para facilitar el acceso a todas las opciones disponibles.

RF-13 El programa validará los datos ingresados por el usuario mostrando mensajes de error cuando exista información inválida.

REQUERIMIENTOS NO FUNCIONALES

- RNF-01** El proyecto será desarrollado utilizando DEV C++ en un entorno Windows.
- RNF-03** Las estructuras deberán implementarse manualmente mediante nodos y punteros, sin utilizar librerías predefinidas.
- RNF-05** El código fuente deberá mantenerse organizado mediante funciones y módulos para facilitar su comprensión y mantenimiento.
- RNF-06** El sistema contará con almacenamiento permanente de datos mediante archivos de texto.
- RNF-08** El código fuente será almacenado dentro de un repositorio Git para mantener el control de versiones del proyecto.
- RNF-09** El proyecto incluirá documentación técnica en formato PDF explicando el funcionamiento del sistema.

1.3 ESTRUCTURAS DE DATOS PROPUESTAS

En el sistema se utilizan estructuras de datos dinámicas lineales, ya que permiten manejar la información de manera flexible durante la ejecución del programa. A diferencia de las estructuras estáticas, no requieren un tamaño fijo, lo que facilita agregar o eliminar elementos según sea necesario sin desperdiciar memoria.

1.

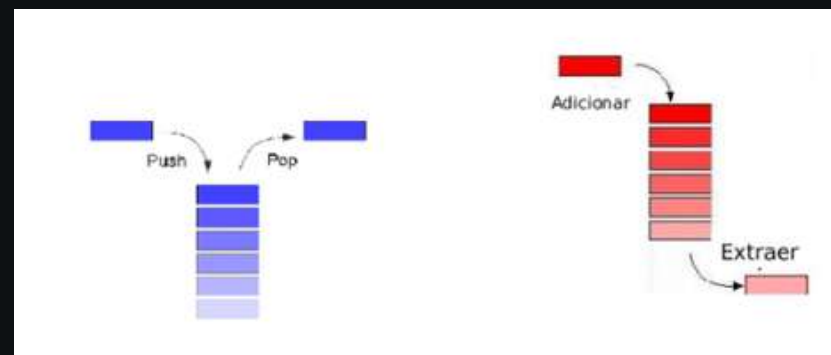
Lista Enlazada Simple (Gestor de Procesos):

Permite registrar procesos de forma dinámica mediante nodos conectados por punteros. Facilita operaciones como insertar, eliminar, buscar y recorrer procesos, adaptándose a los cambios constantes del sistema.

02.

Cola de Prioridad (Planificador de CPU):

Organiza los procesos según su nivel de prioridad, asegurando que los más importantes se ejecuten primero. Permite encolar, desencolar y visualizar el estado de ejecución.



03.

Pila - Stack (Gestor de Memoria):

Utiliza el principio LIFO (último en entrar, primero en salir) para representar la asignación y liberación de memoria. Permite controlar los bloques de memoria mediante operaciones push y pop.

04.

Pila - Stack (Gestor de Memoria):

Utiliza una función matemática basada en el residuo para convertir el ID del proceso en un índice numérico directo. Permite optimizar la velocidad de localización de las tareas mediante operaciones de inserción, búsqueda inmediata y eliminación indexada.

1.4 JUSTIFICACIÓN DE LA ELECCIÓN

Las estructuras seleccionadas permiten representar de forma más realista el funcionamiento básico de un sistema operativo, además de mejorar la organización y eficiencia en el manejo de datos.

Lista Enlazada:

Se eligió porque permite gestionar procesos de manera flexible sin necesidad de un tamaño fijo, facilitando la inserción, eliminación y modificación de datos en cualquier momento.

Cola de Prioridad:

Fue seleccionada porque simula adecuadamente la planificación de CPU, permitiendo que los procesos se ejecuten no solo por orden de llegada, sino también por su importancia.

Pila:

Se utiliza porque su funcionamiento LIFO coincide con la forma en que se gestionan ciertos recursos en memoria, permitiendo un control ordenado de la asignación y liberación.

Además, el sistema hace uso de:

- Punteros, para manejar memoria dinámica y enlazar nodos.
- Funciones, para estructurar y organizar el código.
- Estructuras de control como while, do-while, for, switch e if

CAPITULO 2

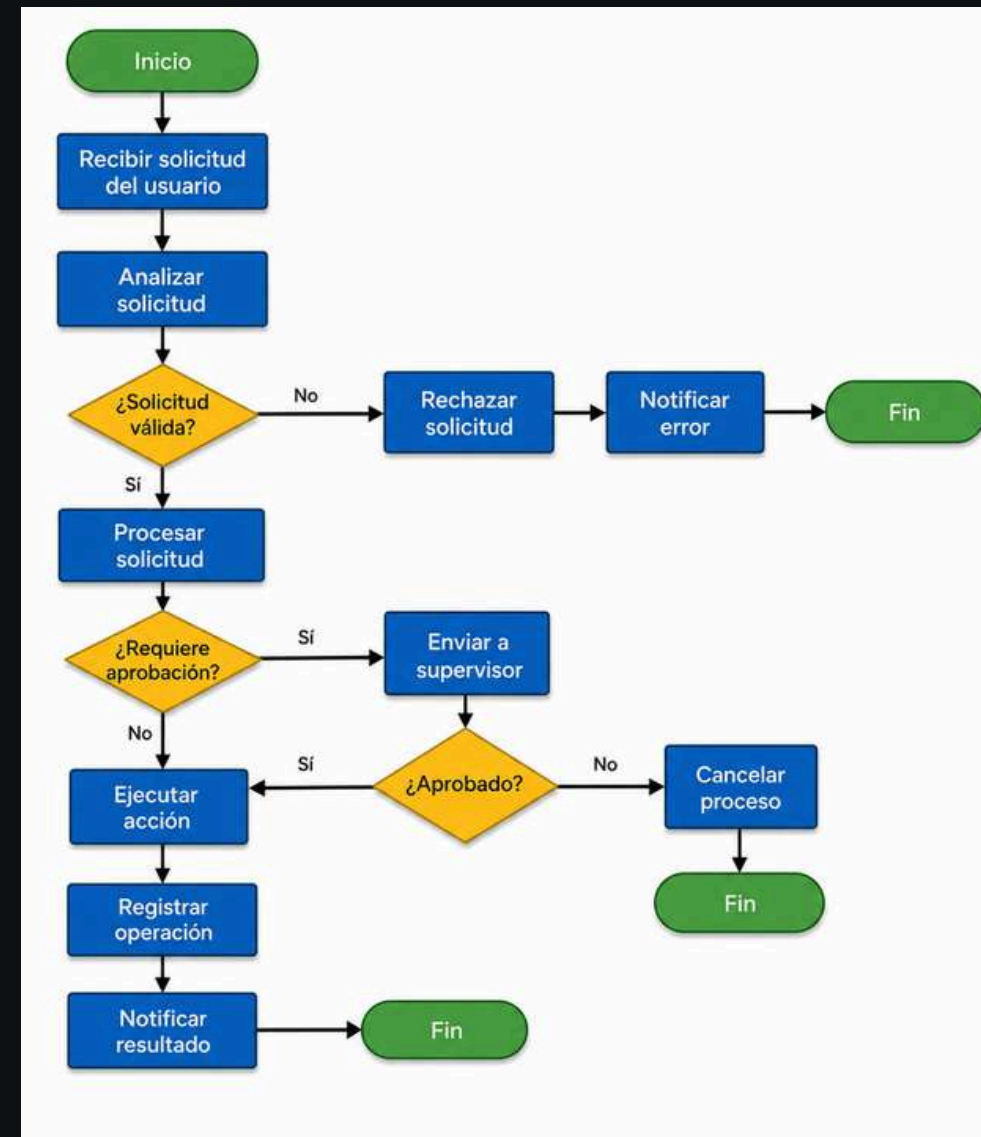


2.3 DIAGRAMAS DE FLUJO DEL SISTEMA

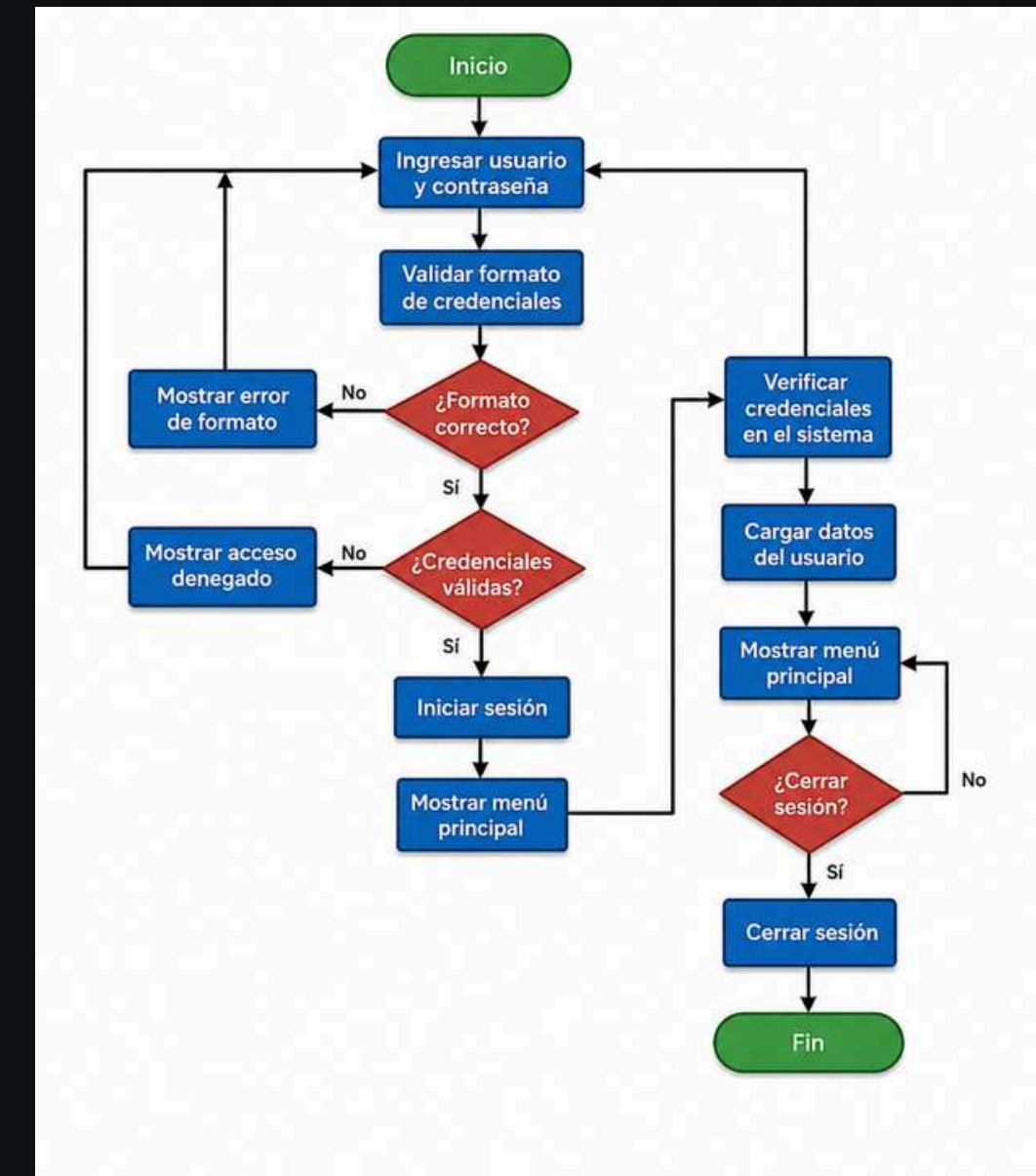
Diagramas de Flujo del Sistema de Gestión de Procesos



GESTOR DE
PROCESOS



PLANIFICADOR
DE CPU



GESTOR DE
MEMORIA

2.3 DIAGRAMAS DE FLUJO DEL SISTEMA

Descripción general

Los diagramas de flujo representan gráficamente el funcionamiento del Sistema de Gestión de Procesos. Cada módulo fue diseñado para realizar una tarea específica dentro del sistema, permitiendo organizar mejor las operaciones y facilitar el control de los procesos.

Módulos representados

- Gestor de Procesos: administra el registro, validación y actualización de información de los usuarios.
- Planificador de CPU: analiza las solicitudes recibidas, valida su estado y ejecuta las acciones correspondientes.
- Gestor de Memoria: controla el acceso al sistema mediante la validación de credenciales y la gestión de sesiones de usuario.

Beneficios

- Facilitan la comprensión de la lógica del sistema.
- Permiten identificar decisiones y validaciones.
- Ayudan a detectar errores antes de la implementación.



2.4 JUSTIFICACIÓN DEL DISEÑO DEL SISTEMA

El Sistema de Gestión de Procesos fue diseñado utilizando una estructura modular, donde cada componente cumple una función específica. Esta organización permite mejorar la comprensión del sistema, facilitar el mantenimiento del código y simplificar futuras mejoras.

Para el desarrollo de los algoritmos se emplearon estructuras condicionales y repetitivas, las cuales permiten validar correctamente la información ingresada por el usuario. Estas validaciones ayudan a prevenir errores como datos incompletos, identificadores inválidos o procesos inexistentes.

Asimismo, se utilizó pseudocódigo para representar de manera clara la lógica de cada módulo antes de su implementación en C++, permitiendo identificar posibles errores durante la etapa de diseño.

Finalmente, los diagramas de flujo sirvieron como apoyo visual para representar el comportamiento del sistema, facilitando el análisis, la comprensión y la posterior programación de cada proceso.

CAPITULO 3



**Código limpio,
bien comentado y
estructurado.**



PROS Y CONTRAS

Pros:

- Optimizar procesos.
- Mejorar las tareas.
- Menor carga de trabajo.
- Mejorar la productividad.
- Aumentar las ventas.
- Nuevas oportunidades.
- Reducir gasto en recursos.



Contras:

- Tiempo de aprendizaje.
- Inversión económica.
- Gasto en recursos.
- Inversión en marketing.
- Rediseño de productos.
- Inversión en redes sociales.

**Capturas de pantalla de
las ventanas de
ejecución con las
diversas pruebas de
validación de datos**



1. Registrar Proceso:

Te pide los datos de un nuevo proceso. Valida que el ID sea positivo, que no esté repetido y que la prioridad sea del 1 al 5. Si todo está bien, lo guarda al final de la lista, lo activa y lo indexa en la tabla Hash.

```
0. Salir
Seleccione una opcion: 1

Ingrese ID del proceso: |
```

2. Mostrar Proceso:

Revisa si hay procesos guardados en el sistema. Si la lista está vacía, te avisa; si no, recorre toda la lista desde el principio y te muestra en pantalla los datos de cada proceso en orden.

```
0. Salir
Seleccione una opcion: 2

No hay procesos registrados.
```

3. Buscar Proceso

Te pide el ID que quieres encontrar. Primero va directo a la tabla Hash para buscarlo al instante; si no lo encuentra ahí por algún choque de datos, busca uno por uno en la lista principal hasta dar con él o avisarte que no existe.

```
Ingrese ID a buscar: |
```

4. Modificar prioridad

Busca un proceso por su ID. Si existe, te permite escribirle una nueva prioridad, asegurándose de que esté en el rango del 1 al 5. Si no pones un número válido o el proceso no existe, cancela el cambio.

```
3 Seleccione una opcion: 4
9
0 Ingrese ID del proceso: |
```

5. Cambiar Estado:

Te pide el ID de un proceso para buscarlo. Si lo encuentra, te deja escribir con texto su nueva situación actual (por ejemplo: si pasó de estar "Activo" a estar "Bloqueado" o "Pausado").

```
o. 3a11
Seleccione una opcion: 5
Ingrese ID del proceso: |
```

6. Eliminar Proceso:

Busca el proceso por su ID. Cuando lo encuentra, lo desconecta de la lista con cuidado para no romper la cadena, borra su rastro de la tabla Hash y destruye el nodo por completo para liberar el espacio que ocupaba en la memoria de la computadora.

```
Seleccione una opcion: 6
Ingrese ID del proceso a eliminar: |
```

7. Encolar proceso en CPU:

El sistema valida que el ID ingresado exista previamente en la lista de procesos. Si el proceso no se encuentra registrado, se muestra el mensaje "Proceso no encontrado" y la operación de encolado es cancelada.

```
Seleccione una opcion: 7
Ingrese ID del proceso para enviar a CPU: |
```

8. Ejecutar proceso CPU:

El sistema verifica si la cola de CPU contiene procesos pendientes. Cuando la cola está vacía, muestra el mensaje "La cola de CPU está vacía" e impide la ejecución de procesos inexistentes.

```
Seleccione una opcion: 8
```

```
La cola de CPU esta vacia.
```

9. Mostrar cola CPU:

El sistema verifica si existen procesos registrados en la cola de CPU antes de mostrarlos. Cuando la cola está vacía, se informa al usuario mediante el mensaje "La cola de CPU está vacía".

```
Seleccione una opcion: 9
```

```
La cola de CPU esta vacia.
```

10. Asignar memoria PUSH

El sistema valida que el ID del proceso exista antes de realizar la operación PUSH. Si el proceso no se encuentra registrado, no se asigna memoria y se muestra un mensaje de error, evitando almacenar bloques de memoria asociados a procesos inexistentes.

```
Seleccione una opcion: 10
```

```
Ingrese ID del proceso: 233
```

```
Error: el proceso no existe.
```

11. Liberar memoria POP

El sistema verifica que existan bloques de memoria almacenados en la pila antes de realizar la operación POP. Si no hay memoria asignada, la liberación es cancelada para evitar errores y se muestra un mensaje informando que la pila está vacía.

```
Seleccione una opcion: 11
```

```
La pila de memoria esta vacia.
```

12. Mostrar pila de memoria

El sistema verifica si existen bloques de memoria almacenados en la pila antes de mostrarlos. Cuando no hay memoria asignada, se informa al usuario mediante el mensaje "No hay memoria asignada en la pila".

```
Seleccione una opcion: 12
```

```
No hay memoria asignada en la pila.
```

14. Guardar datos:

El sistema almacena la información de los procesos registrados en el archivo "procesos.txt", permitiendo conservar los datos para futuras ejecuciones del programa.

```
Seleccione una opcion: 14
```

```
Datos guardados en procesos.txt
```

13. Mostrar tabla hash

El sistema valida si existen procesos almacenados en la tabla hash antes de mostrarlos. Cuando no hay procesos registrados, todas las posiciones aparecen vacías, indicando que la estructura no contiene información.

```
Seleccione una opcion: 13
```

```
=== TABLA HASH DE PROCESOS ===
```

```
Posicion [0]: Vacio
```

```
Posicion [1]: Vacio
```

```
Posicion [2]: Vacio
```

```
Posicion [3]: Vacio
```

```
Posicion [4]: Vacio
```

```
Posicion [5]: Vacio
```

```
Posicion [6]: Vacio
```

```
Posicion [7]: Vacio
```

```
Posicion [8]: Vacio
```

```
Posicion [9]: Vacio
```

MANUAL DE USUARIO



1. Gestión de Procesos

Permite registrar, visualizar, buscar, modificar y eliminar procesos.

Cada proceso tiene ID único, nombre, prioridad (1-5) y estado.

El estado inicial es "Activo".

```
===== SISTEMA DE GESTION DE PROCESOS =====
```

1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso

```
=== LISTA DE PROCESOS ===
```

ID: 10		Nombre: Chrome		Prioridad: 3		Estado: Activo
ID: 20		Nombre: Discord		Prioridad: 1		Estado: Activo
ID: 30		Nombre: VSCode		Prioridad: 5		Estado: Activo
ID: 40		Nombre: Spotify		Prioridad: 4		Estado: Activo



2. Planificación de CPU

Simula la ejecución de procesos según prioridad.

Los procesos se encolan automáticamente por prioridad (mayor primero).

La CPU ejecuta el proceso al frente de la cola.

7. Encolar proceso en CPU

8. Ejecutar proceso CPU

9. Mostrar cola CPU

seleccione una opcion: 9

== COLA DE PRIORIDAD CPU ==

ID: 40	Nombre: Spotify	Prioridad: 4
ID: 40	Nombre: Spotify	Prioridad: 4
ID: 20	Nombre: Discord	Prioridad: 1

3. Control de Memoria RAM

Usa estructura tipo pila (LIFO).
El último bloque agregado es el primero en eliminarse.

Permite asignar, liberar y visualizar memoria.

10. Asignar memoria PUSH
11. Liberar memoria POP

```
===== SISTEMA DE GESTION DE PROCESOS =====
```

```
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
```

```
Seleccione una opcion: 11
```

```
Liberando memoria (POP):
```

```
ID Proceso: 10 | Bloque: 103 | Tamano: 128
```

```
Memoria liberada correctamente.
```


4. Almacenamiento y Sistema

Incluye tabla hash para búsqueda rápida.

Permite guardar datos en archivo.

Salida segura guarda información y libera memoria.

13. Mostrar tabla hash

14. Guardar datos

0. Salir

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 13

=== TABLA HASH DE PROCESOS ===
Posicion [0]: ID 40 - Spotify
Posicion [1]: Vacio
Posicion [2]: Vacio
Posicion [3]: Vacio
```



RECOMENDACIONES

5. Flujo de Uso Recomendado

Registrar proceso → Encolar en CPU → Ver cola → Ejecutar proceso.

6. Consideraciones

No usar IDs inexistentes.
Salir correctamente para evitar pérdida de datos.

¡ Gracias !

